Day 4 - String Pattern Match

ITSRUNTYM

Pattern Matching

- **KMP Algorithm** (Knuth-Morris-Pratt)

1. Naive Pattern Matching
Idea
Check each possible position in the text and match character-by-character with the pattern.

Time Complexity: O((n-m+1) * m)
Example

Text : ababcabcab
Pattern : abc
We check at every index if pattern matches.

Code

```
public int naiveMatch(String text, String pattern) {
    int n = text.length();
    int m = pattern.length();

   for (int i = 0; i <= n - m; i++) {
        int j = 0;
        while (j < m && text×charAt(i + j) == pattern.charAt(j)) {
            j++;
        }
        if (j == m) return i; // match found
    }
    return -1;
}
```

C++

```
int naiveMatch(const string& text, const string& pattern) {
    int n = text.length();
    int m = pattern.length();

    for (int i = 0; i <= n - m; i++) {
        int j = 0;

        // Compare characters of pattern and text at current index
        while (j < m && text[i + j] == pattern[j]) {
            j++;
        }

        if (j == m) {
            return i;  // Pattern found at index i
        }
    }
    return -1; // No match found
}
```

2. KMP Algorithm (Knuth-Morris-Pratt)

What is KMP?

Problem: Find the first occurrence of a pattern in a string.

Naive approach compares character by character from each index → **O(n * m)** in worst case.

KMP Goal: Avoid unnecessary comparisons by remembering past matches.

Key Concept – LPS Array

LPS[i]: Longest proper prefix of pattern[0..i] that is also a suffix.
Proper prefix = not equal to the full string.
It tells us how many characters we can skip when a mismatch happens.

LPS Table Dry Run
Pattern = "abcab"
We want to build this:

• Index	• 0	• 1	• 2	• 3	• 4
• Char	• a	• b	• c	• a	• b
• LPS	• 0	• 0	• 0	• 1	• 2
• **Steps:**					

    - Start from i = 1, len = 0
    - If pattern[i] == pattern[len] → lps[i] = len+1, len++
    - Else → if len != 0, reduce len = lps[len-1], else lps[i] = 0
- So:
    - i=1: b != a → lps[1]=0
    - i=2: c != a → lps[2]=0
    - i=3: a == a → lps[3]=1
    - i=4: b == b → lps[4]=2
- **Code Explanation**

```
public int strStr(String text, String pattern) {
    if (pattern.isEmpty()) return 0;
    int[] lps = computeLPS(pattern); // Step 1: Build LPS
    int i = 0, j = 0;

    while (i < text.length()) {
        if (text×charAt(i) == pattern.charAt(j)) {
            i++; j++;
        }

        if (j == pattern.length()) return i - j; // match found

        else if (i < text.length() && text×charAt(i) != pattern.charAt(j)) {
            if (j != 0) j = lps[j - 1]; // jump to previous possible match
            else i++; // if j == 0, no prefix match, just move text index
        }
    }
    return -1;
}
```

LPS Building Code:

```
private int[] computeLPS(String pat) {
    int[] lps = new int[pat.length()];
    int len = 0, i = 1;
```

```
while (i < pat.length()) {
        if (pat×charAt(i) == pat.charAt(len)) {
            len++;
            lps[i++] = len;
        } else {
            if (len != 0) len = lps[len - 1];
            else lps[i++] = 0;
        }
    }
return lps;
}
```

C++

```
class Solution {
public:
    int strStr(string text, string pattern) {
        if (pattern.empty()) return 0;

        vector<int> lps = computeLPS(pattern);
        int i = 0, j = 0;

        while (i < text.length()) {
            if (text[i] == pattern[j]) {
                i++;
                j++;
            }

            if (j == pattern.length()) {
                return i - j; // Pattern found
            } else if (i < text.length() && text[i] != pattern[j]) {
                if (j != 0) {
                    j = lps[j - 1]; // Fall back in pattern using LPS
                } else {
                    i++; // No match, move text index
                }
            }
        }
        return -1; // Pattern not found
    }

private:
    vector<int> computeLPS(string pattern) {
```

```
        int n = pattern.length();
        vector<int> lps(n, 0);

        int len = 0; // length of the previous longest prefix suffix
        int i = 1;

        while (i < n) {
            if (pattern[i] == pattern[len]) {
                len++;
                lps[i] = len;
                i++;
            } else {
                if (len != 0) {
                    len = lps[len - 1]; // Try shorter previous lps
                } else {
                    lps[i] = 0;
                    i++;
                }
            }
        }

        return lps;
    }
};
```